

SALUS SECURITY

JUN 2026



CODE SECURITY ASSESSMENT

TRADE VAULT

Overview

Project Summary

- Name: TradeVault
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
 - <https://github.com/dwilminkone8/TradeVault>
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Trade Vault
Version	v2
Type	Solidity
Dates	Jun 03 2026
Logs	Jun 02 2026, Jun 03 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	2
Total Low-Severity issues	4
Total informational issues	2
Total	8

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Executors can manipulate slippage to earn profit via MEV	6
2. Centralization risk	7
3. Native token swap is not supported	8
4. Missing revoke approval when disabling permit2	9
5. Executors can execute unintended router operations via arbitrary calldata	10
6. Executors can transfer vault assets to arbitrary recipients through router calldata	11
2.3 Informational Findings	12
7. Mapping Declaration Lacks Named Parameters for Improved Readability	12
8. Lack of Security Contact Information for Responsible Disclosure	13
Appendix	14
Appendix 1 - Files in Scope	14

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Executors can manipulate slippage to earn profit via MEV	Medium	Business Logic	Acknowledged
2	Centralization risk	Medium	Centralization	Acknowledged
3	Native token swap is not supported	Low	Business Logic	Acknowledged
4	Missing revoke approval when disabling permit2	Low	Configuration	Acknowledged
5	Executors can execute unintended router operations via arbitrary calldata	Low	Business Logic	Acknowledged
6	Executors can transfer vault assets to arbitrary recipients through router calldata	Low	Business Logic	Acknowledged
7	Mapping Declaration Lacks Named Parameters for Improved Readability	Informational	Business Logic	Acknowledged
8	Lack of Security Contact Information for Responsible Disclosure	Informational	Business Logic	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Executors can manipulate slippage to earn profit via MEV

Severity: Medium

Category: Business Logic

Target:

- TradeVault.sol

Description

In TradeVault, the function `swapViaRouter` is used by the whitelist executors to swap the existing assets into one expected asset.

The vault does not independently validate whether the swap is executed within an acceptable price range. A malicious or compromised executor may submit a swap with insufficient slippage protection, such as setting the minimum output amount to zero. This may allow the executor or another MEV actor to manipulate the pool price, execute the vault's trade at an unfavorable rate, and capture the vault's loss through a sandwich strategy.

TradeVault.sol:L305-L321

```
function swapViaRouter(  
    address _router,  
    bytes calldata _callData,  
    address payable _tipTo  
) external payable onlyExecutor whenNotPaused nonReentrant {  
    if (!whitelistedRouters[_router]) revert RouterNotWhitelisted(_router);  
    ...  
}
```

Recommendation

Consider calculating the total asset value and make sure that the swap action cannot decrease the whole vault's asset value too much.

Status

This issue has been acknowledged by the team. The team stated that the executor is a trusted third party.

2. Centralization risk

Severity: Medium

Category: Business Logic

Target:

- TradeVault.sol

Description

In the TradeVault contract, there exists one privileged role, `owner`. This role has the authority to execute some key functions such as `withdraw`.

If these roles' private keys are compromised, an attacker could trigger these functions to withdraw all funds.

TradeVault.sol:L372-L385

```
function withdrawToken(address _token, address _to, uint256 _amount) external onlyOwner
nonReentrant {
    if (_to == address(0)) revert ZeroAddress();
    IERC20(_token).safeTransfer(_to, _amount);
    emit TokenWithdrawn(_token, _to, _amount);
}

/// @notice Withdraw native ETH to an arbitrary address.
function withdrawNative(address payable _to, uint256 _amount) external onlyOwner
nonReentrant {
    if (_to == address(0)) revert ZeroAddress();
    if (address(this).balance < _amount) revert InsufficientBalance();
    (bool sent, ) = _to.call{value: _amount}("");
    if (!sent) revert TransferFailed();
    emit NativeWithdrawn(_to, _amount);
}
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been acknowledged by the team.

3. Native token swap is not supported

Severity: Low

Category: Centralization

Target:

- TradeVault.sol

Description

In TradeVault, the function `swapViaRouter` is used by the whitelist executors to swap the existing assets into one expected asset.

The tradeVault is expected to be compatible with any DEX router. Some DEX routers support native token swap. The problem here is that `_router.call` does not support native token swap.

TradeVault.sol:L305-L321

```
function swapViaRouter(  
    address _router,  
    bytes calldata _callData,  
    address payable _tipTo  
) external payable onlyExecutor whenNotPaused nonReentrant {  
    if (!whitelistedRouters[_router]) revert RouterNotWhitelisted(_router);  
  
    (bool success, bytes memory returnData) = _router.call(_callData);  
    if (!success) {  
        assembly ("memory-safe") {  
            revert(add(returnData, 32), mload(returnData))  
        }  
    }  
    emit SwapExecuted(_router, _callData.length);  
  
    if (msg.value > 0) _tip(_tipTo, msg.value);  
}
```

Recommendation

Support the native token swap if needed.

Status

This issue has been resolved by the team with commit [c32ea22](#).

4. Missing revoke approval when disabling permit2

Severity: Low

Category: Centralization

Target:

- TradeVault.sol

Description

In TradeVault, the function `swapViaRouter` is used by the whitelist executors to swap the existing assets into one expected asset.

When the permit2 is added into the whitelist, the maximum approval allowance will be set in the function `approvePermit2Router`.

The problem here is that when the permit2 is disabled, the function misses to revoke the maximum approval allowance.

TradeVault.sol:L183-L188

```
function setPermit2(address _addr, bool _active) external onlyOwner {
    if (_addr == address(0)) revert ZeroAddress();
    if (_active && (_addr == WETH || whitelistedRouters[_addr])) revert
InvalidPermit2(_addr);
    whitelistedPermit2[_addr] = _active;
    emit Permit2Set(_addr, _active);
}
```

TradeVault.sol:L243-L259

```
function approvePermit2Router(
    address _permit2,
    address _token,
    address _router,
    uint160 _amount,
    uint48 _expiration
) external onlyOwner {
    if (_amount > 0) {
        if (!whitelistedPermit2[_permit2]) revert Permit2NotWhitelisted(_permit2);
        if (!whitelistedRouters[_router]) revert RouterNotWhitelisted(_router);
        // approve uint256.max to permit2.
        IERC20(_token).forceApprove(_permit2, type(uint256).max);
    }
    // permit2 approve router.
    IPermit2(_permit2).approve(_token, _router, _amount, _expiration);
    emit Permit2RouterApproved(_permit2, _token, _router, _amount, _expiration);
}
```

Recommendation

Revoke approval allowance when one permit2 is disabled.

Status

This issue has been resolved by the team with commit [c32ea22](#).

5. Executors can execute unintended router operations via arbitrary calldata

Severity: Low

Category: Business Logic

Target:

- TradeVault.sol

Description

The `swapViaRouter` function is intended to allow whitelisted executors to perform token swaps through approved routers. However, the vault only verifies that the router address is whitelisted and forwards executor-controlled calldata without validating the called function or its parameters.

Many router contracts expose operations other than swaps, such as adding liquidity or executing multicall commands. As a result, an executor may invoke router functionality that does not match the intended swap behavior of the vault.

This issue may also enable more direct fund-loss paths depending on the selected router operation and its parameters. For example, an executor may add liquidity while assigning the resulting LP tokens to an external address.

TradeVault.sol:L305-L321

```
function swapViaRouter(  
    address _router,  
    bytes calldata _callData,  
    address payable _tipTo  
) external payable onlyExecutor whenNotPaused nonReentrant {  
    if (!whitelistedRouters[_router]) revert RouterNotWhitelisted(_router);  
    ...  
}
```

Recommendation

Use router-specific adapters that restrict the allowed function selectors and validate decoded parameters. Ensure each permitted operation matches the vault's intended trading workflow.

Status

This issue has been acknowledged by the team. The team stated that the executor is a trusted third party.

6. Executors can transfer vault assets to arbitrary recipients through router calldata

Severity: Low

Category: Business Logic

Target:

- TradeVault.sol

Description

The `swapViaRouter` function forwards executor-controlled calldata to an approved router without validating the invoked function, command sequence, or recipient addresses.

A malicious or compromised executor may transfer vault assets to an arbitrary recipient through different paths:

a. Assigning an arbitrary swap output recipient

Many routers allow the caller to specify the recipient of the swap output. An executor may submit an otherwise valid swap while assigning an attacker-controlled address as the receiver.

The router pulls the input tokens from the vault, but sends the output tokens directly to the attacker-controlled address.

b. Executing Permit2-based token transfer commands

Some routers, such as Universal Router implementations, support command sequences that use Permit2 allowances to transfer tokens. If the vault has granted the relevant allowances, an executor may submit a command sequence that pulls approved tokens from the vault and transfers them to an attacker-controlled address without performing the expected swap.

Both paths bypass the recipient restrictions enforced by `transferToken()` and may allow an executor to extract vault assets.

TradeVault.sol:L305-L321

```
function swapViaRouter(  
    address _router,  
    bytes calldata _callData,  
    address payable _tipTo  
) external payable onlyExecutor whenNotPaused nonReentrant {  
    if (!whitelistedRouters[_router]) revert RouterNotWhitelisted(_router);  
    ...  
}
```

Recommendation

Consider calculating the total asset value and make sure that the swap action cannot decrease the whole vault's asset value too much.

Status

This issue has been acknowledged by the team. The team stated that the executor is a trusted third party.

2.3 Informational Findings

7. Mapping Declaration Lacks Named Parameters for Improved Readability

Severity: Informational

Category: Redundancy

Target:

- TradeVault.sol

Description

Since Solidity version 0.8.18, the language allows developers to add named key and value identifiers in mapping declarations using the syntax:

```
mapping(KeyType keyName => ValueType valueName)
```

This enhances code readability and clarifies how a mapping is intended to be used. In the TradeVault contract, the mapping executors, whitelistedRouters, etc are declared without named parameters, making the purpose of the key and value less obvious upon inspection.

TradeVault.sol:L305-L321

```
contract TradeVault is Ownable2Step, Pausable, ReentrancyGuard {
    using SafeERC20 for IERC20;
    //

    // STATE
    //

    /// @notice Canonical WETH address on this chain. Immutable after deployment.
    address public immutable WETH;
    /// @notice Addresses authorised to call swap, transfer, wrap/unwrap functions.
    // @audit-info Mapping Declaration Lacks Named Parameters for Improved Readability
    mapping(address => bool) public executors;
    /// @notice Router contracts executors may relay arbitrary swap calldata to.
    mapping(address => bool) public whitelistedRouters;
    /// @notice Permit2 contracts the vault may interact with for two-step approvals.
    mapping(address => bool) public whitelistedPermit2;
    /// @notice Addresses to which executors may push tokens or ETH via
    transferToken/transferNative.
    mapping(address => bool) public whitelistedDepositAddresses;
    /// @notice Addresses to which executors may send ETH tips (e.g. block.coinbase,
    relayers).
    mapping(address => bool) public whitelistedTipRecipients;
}
```

Recommendation

Consider updating the mapping declaration to explicitly name the key and value identifiers, enhancing readability and making the mapping's purpose clearer.

Status

This issue has been resolved by the team with commit [c32ea22](#).

8. Lack of Security Contact Information for Responsible Disclosure

Severity: Informational

Category: Redundancy

Target:

- TradeVault.sol

Description

The contract TradeVault does not specify a security contact point. Including a designated security contact (such as an email address or ENS name) in the contract's NatSpec header facilitates responsible vulnerability disclosure. This makes it easier for external researchers to quickly reach the appropriate team in the event a vulnerability is identified, helping minimize the time window between discovery and mitigation. The Ethereum community has begun standardizing this practice using the `@custom:security-contact` tag, adopted by tools such as OpenZeppelin Wizard and ethereum-lists.

Recommendation

Consider adding a NatSpec comment at the top of the contract with a `@custom:security-contact` field pointing to the preferred disclosure channel.

Status

This issue has been resolved by the team with commit [c32ea22](#).

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit [d8338ca](#):

File	SHA-1 hash
TradeVault.sol	5fec568677fe79ddf4238aa9570d69d39bc2479c